API Routes

If your application will also offer a stateless API, you may enable API routing using the `install:api` Artisan command:

```
php artisan install:api
```

The `install:api` command installs Laravel Sanctum, which provides a robust, yet simple API token authentication guard which can be used to authenticate third-party API consumers, SPAs, or mobile applications. In addition, the `install:api` command creates the `routes/api.php` file:

```
Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');
```

****

The routes in `routes/api.php` are stateless and are assigned to the `api` middleware group. Additionally, the `/api` URI prefix is automatically applied to these routes, so you do not need to manually apply it to every route in the file. You may change the prefix by modifying your application's `bootstrap/app.php` file:

```
->withRouting(
    api: __DIR__.'/../routes/api.php',
    apiPrefix: 'api/admin',
    // ...
)
```

****

Sometimes you may need to register a route that responds to multiple HTTP verbs. You may do so using the `match` method. Or, you may even register a route that responds to all HTTP verbs using the `any` method:

```
Route::match(['get', 'post'], '/', function () {
    // ...
});

Route::any('/', function () {
    // ...
});
```

****

Redirect Routes

If you are defining a route that redirects to another URI, you may use the `Route::redirect` method. This method provides a convenient shortcut so that you do not have to define a full route or controller for performing a simple redirect:

```
Route::redirect('/here', '/there');
```

By default, `Route::redirect` returns a `302` status code. You may customize the status code using the optional third parameter:

```
Route::redirect('/here', '/there', 301);
```

****

By default, `Route::redirect` returns a `302` status code. You may customize the status code using the optional third parameter:

```
Route::redirect('/here', '/there', 301);
```

Or, you may use the `Route::permanentRedirect` method to return a `301` status code:

```
Route::permanentRedirect('/here', '/there');
```

> When using route parameters in redirect routes, the following parameters are reserved by Laravel and cannot be used: `destination` and `status`.

****

View Routes

If your route only needs to return a view, you may use the `Route::view` method. Like the `redirect` method, this method provides a simple shortcut so that you do not have to define a full route or controller. The `view` method accepts a URI as its first argument and a view name as its second argument. In addition, you may provide an array of data to pass to the view as an optional third argument:

```
Route::view('/welcome', 'welcome');

Route::view('/welcome', 'welcome', ['name' => 'Taylor']);
```

> When using route parameters in view routes, the following parameters are reserved by Laravel and cannot be used: `view`, `data`, `status`, and `headers`.

****

Listing Your Routes

The `route:list` Artisan command can easily provide an overview of all of the routes that are defined by your application:

```
php artisan route:list
```

By default, the route middleware that are assigned to each route will not be displayed in the `route:list` output; however, you can instruct Laravel to display the route middleware and middleware group names by adding the `-v` option to the command:

```
php artisan route:list -v

# Expand middleware groups ...
php artisan route:list -vv
```

****

You may also instruct Laravel to only show routes that begin with a given URI:

```
php artisan route:list --path=api
```

In addition, you may instruct Laravel to hide any routes that are defined by third-party packages by providing the `--except-vendor` option when executing the `route:list` command:

```
php artisan route:list --except-vendor
```

Likewise, you may also instruct Laravel to only show routes that are defined by third-party packages by providing the `--only-vendor` option when executing the `route:list` command:

```
php artisan route:list --only-vendor
```

****

```php
<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use Illuminate\Support\Facades\Route;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
        then: function() {
            Route::middleware('api')
            ->prefix('webhooks')
            ->name('webhooks.')
            ->group(base_path('routes/webhooks.php'));
        }
    )
    ->withMiddleware(function (Middleware $middleware): void {
        //
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```

****

Or, you may even take complete control over route registration by providing a `using` closure to the `withRouting` method. When this argument is passed, no HTTP routes will be registered by the framework and you are responsible for manually registering all routes:

```php
use Illuminate\Support\Facades\Route;

->withRouting(
    commands: __DIR__.'/../routes/console.php',
    using: function () {
        Route::middleware('api')
            ->prefix('api')
            ->group(base_path('routes/api.php'));

        Route::middleware('web')
            ->group(base_path('routes/web.php'));
    },
)
```

****

Global Constraints

If you would like a route parameter to always be constrained by a given regular expression, you may use the `pattern` method. You should define these patterns in the `boot` method of your application's `App\Providers\AppServiceProvider` class:

```php
use Illuminate\Support\Facades\Route;

/**
 * Bootstrap any application services.
 */
public function boot(): void
{
    Route::pattern('id', '[0-9]+');
}
```

****

In order to ensure your subdomain routes are reachable, you should register subdomain routes before registering root domain routes. This will prevent root domain routes from overwriting subdomain routes which have the same URI path.

****

Customizing the Key

Sometimes you may wish to resolve Eloquent models using a column other than `id`. To do so, you may specify the column in the route parameter definition:

```php
use App\Models\Post;

Route::get('/posts/{post:slug}', function (Post $post) {
    return $post;
});
```

****

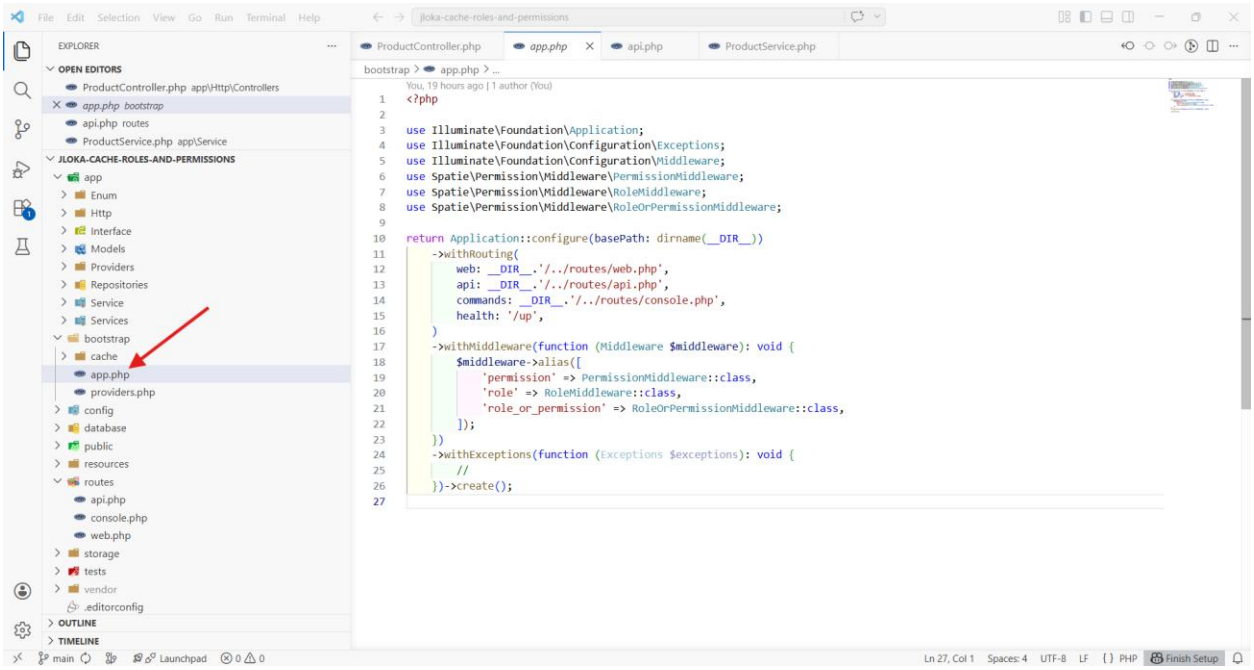
If you would like model binding to always use a database column other than `id` when retrieving a given model class, you may override the `getRouteKeyName` method on the Eloquent model:

```php
/**
 * Get the route key for the model.
 */
public function getRouteKeyName(): string
{
    return 'slug';
}
```

****

To use Spatie Permission and role middleware in a Laravel Application:

```php
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->alias([
            'permission' => PermissionMiddleware::class,
            'role' => RoleMiddleware::class,
            'role_or_permission' => RoleOrPermissionMiddleware::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```



****

When we create a seeder for a model, and the model has relationships, then we can use the has-method with the model factory to generate the data:

Ex: User::factory(50)->has(Vehicle::factory()->count(3))->create()

****

Installation

Octane may be installed via the Composer package manager:

```
composer require laravel/octane
```

After installing Octane, you may execute the `octane:install` Artisan command, which will install Octane's configuration file into your application:

```
php artisan octane:install
```

****

```
./vendor/bin/sail up

./vendor/bin/sail composer require laravel/octane
```

Next, you should use the `octane:install` Artisan command to install the FrankenPHP binary:

```
./vendor/bin/sail artisan octane:install --server=frankenphp
```

Finally, add a `SUPERVISOR_PHP_COMMAND` environment variable to the `laravel.test` service definition in your application's `docker-compose.yml` file. This environment variable will contain the command that Sail will use to serve your application using Octane instead of the PHP development server:

```
services:
  laravel.test:
    environment:
      SUPERVISOR_PHP_COMMAND: "/usr/bin/php -d variables_order=EGPCS /va
      XDG_CONFIG_HOME:  /var/www/html/config
      XDG_DATA_HOME:  /var/www/html/data
```